

Le Fantôme de la machine

- **Limite de temps** : 10 minutes
- **Score de référence** : 28.6
- **Environnement** : un GPU (≈ 16 GB VRAM), sans internet
- **Taille de la solution** : `solution.ipynb` ≤ 20 MB
- **Stockage** : 5 GB
- **Modèles préentraînés** : uniquement [bge-base-en-v1.5](#) — un **encodeur** de texte (modèle d'embedding).

Tâche

Des événements étranges se produisent aux Archives nationales du Kazakhstan. Les bibliothécaires affirment que certains livres se terminaient autrefois différemment, mais personne ne peut le prouver — tous les exemplaires sont identiques et chaque histoire reste cohérente. Vous êtes invité en tant que chercheur en IA à localiser les modifications.



Un passage commence comme un texte écrit par un humain puis, à un certain moment, passe silencieusement à une continuation générée par un modèle de langage. Lu dans son ensemble, il ressemble à un texte cohérent — mais, quelque part au milieu, l'auteur passe d'une personne à une machine. Votre tâche consiste à **trouver ce changement** : l'indice du caractère où la partie humaine se termine et où la partie générée par la machine commence.

Chaque échantillon est une unique chaîne de caractères `text`. Il existe exactement une frontière. Tout ce qui la précède est humain ; tout ce qui se trouve à partir de celle-ci est généré par une machine.

Dataset

Passages en anglais au format texte brut, avec une frontière chacun.

- **Partie A** (avant la frontière) : un extrait de texte écrit par un humain.
- **Partie B** (à partir de la frontière) : une continuation produite par un modèle de langage, conditionnée par la Partie A.
- Chaque partie contient au moins 180 mots ; la longueur totale est de ~500–800 mots.
- Le `boundary_char_index` est le décalage en caractères où la Partie A se termine : `text[:boundary_char_index]` est la partie humaine et `text[boundary_char_index:].lstrip()` est la partie générée par la machine.

Ce que vous recevez

Vous recevez **deux dossiers** :

Dossier	Échantillons	<code>answers.jsonl</code> ?	Utilisez-le pour
<code>dataset/train/</code>	1,221	☐ inclus	entraîner / affiner votre méthode
<code>dataset/test_public/</code>	380	☐ inclus (copie de développement)	exécuter votre pipeline et calculer vous-même votre score localement

Au **moment de l'évaluation**, votre dossier `dataset/test_public/` est **remplacé par un ensemble d'évaluation caché**. Il a le même format, mais **sans `answers.jsonl`**. Votre notebook est réexécuté sur celui-ci, et le `answers.jsonl` qu'il produit est évalué.

- Le classement public utilise un ensemble caché **test_leaderboard_a** (380 échantillons).
- Le classement final utilise un ensemble caché **test_leaderboard_b** (380 échantillons).

Les trois ensembles d'évaluation ont la même taille et sont tirés de la même distribution que `train` ; votre score `dataset/test_public/` local constitue donc une estimation raisonnable de votre score au classement.

Format sur le disque

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- Les identifiants dans `answers.jsonl` correspondent aux identifiants dans `data.jsonl`.
- `dataset/train/` (avec les réponses) est disponible chaque fois que vous entraînez ou affinez un modèle.

Sortie (format de soumission)

Vous devez soumettre **un unique notebook, qui doit être nommé `solution.ipynb`**. Ce nom de fichier exact est obligatoire. Tout autre fichier est rejeté sans être exécuté.

Votre notebook doit **lire `dataset/test_public/data.jsonl`** et écrire un unique fichier `answers.jsonl` à la racine du dépôt — un objet JSON par ligne, associant chaque identifiant d'échantillon à l'indice de caractère prédit pour la frontière :

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` doit être un **entier dans** `[0, len(text)]`.
- Chaque identifiant dans `dataset/test_public/data.jsonl` doit apparaître exactement une fois. Un échantillon absent de `answers.jsonl` (ou associé à une valeur non entière / hors limites) obtient un score de 0 pour cet échantillon.

Évaluation

Pour chaque échantillon, soit p votre indice prédit et t la vraie frontière. Le score par échantillon décroît exponentiellement avec la distance en caractères :

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

Cela conduit au comportement suivant du score :

- **≈1.0** — caractère exact de la frontière ;
- **≈0.78** — écart de 25 caractères ; - **≈0.61** — écart de 50 caractères ;
- **≈0.37** — écart de 100 caractères ;
- **≈0.01** — écart de 500 caractères.

Le **score final est la moyenne** des scores par échantillon sur tous les échantillons de la partition (exprimée sur une échelle de 0–100). La métrique récompense les prédictions *proches*, et pas seulement les prédictions exactes.

Contraintes

- **Environnement** : un GPU (≈16 GB VRAM), sans internet au moment de l'évaluation — le modèle autorisé (ci-dessous) est déjà fourni. **Budget en temps réel : 10 minutes** pour l'exécution complète — cela doit couvrir tout entraînement / affinage que vous effectuez au moment de l'évaluation, **ainsi que** l'inférence sur l'ensemble d'évaluation.
- **Modèle préentraîné autorisé** — cette liste est exhaustive ; aucun autre poids préentraîné ne peut être utilisé. Il est **fourni à l'avance dans l'environnement** (chargez-le normalement, p. ex. `from_pretrained` ; il n'y a pas d'internet au moment de l'évaluation) :
 - **bge-base-en-v1.5** — un **encodeur** de texte de 110M paramètres (modèle d'embedding). Il produit des embeddings de phrases/passages ; ce n'est pas un modèle de langage génératif. Vous pouvez l'utiliser **tel quel (caractéristiques gelées) ou l'affiner sur la partition `train`** (l'affinage complet respecte le budget de 16 GB / 10 minutes).
- Les outils classiques / statistiques ne sont soumis à aucune restriction : vous pouvez construire n'importe quel modèle fondé sur des caractéristiques (p. ex., des classifieurs ou régresseurs scikit-learn) à partir de caractéristiques d'embedding que vous calculez vous-même. *Les poids de deep learning préentraînés* sont uniquement limités par la liste ci-dessus.